Регуляризация, кросс-валидация и выбор модели

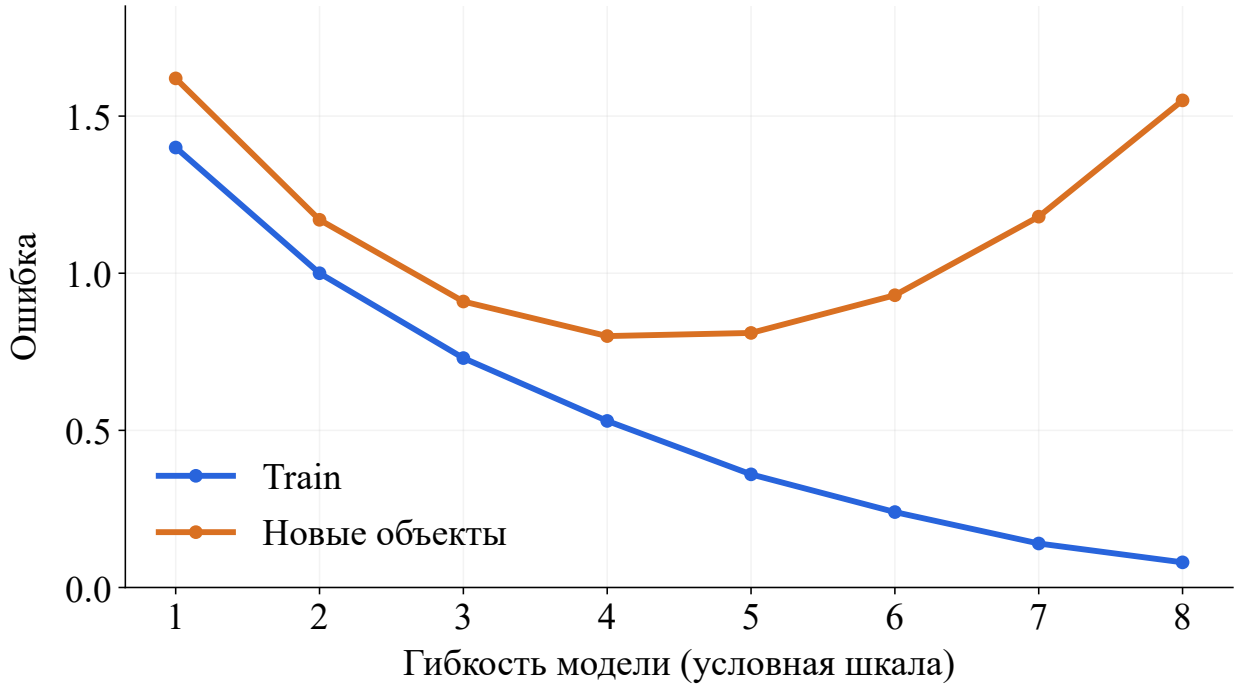
КЛАССИЧЕСКОЕ МАШИННОЕ ОБУЧЕНИЕ · МАГИСТРАТУРА · НЕДЕЛЯ 4

Как ограничить подгонку и выбрать модель,
которая работает на новых объектах?

Регуляризация → **Сравнение по CV** → **Выбор модели**

Логистическая регрессия и kNN · многоклассовая задача

Меньше ошибка на train — лучше новые прогнозы?



Если ошибка на обучении уменьшилась, обязательно ли она уменьшилась на новых наблюдениях?

Схематические кривые: показывают возможное поведение ошибок при росте гибкости. Их конкретная форма зависит от данных и модели.

Связанные признаки: две модели стоимости квартир

Общая площадь — x_1, жилая — x_2. Прогноз — в условных единицах; свободного члена нет.

$$f_A(x) = 3x_1 - 3x_2, \qquad f_B(x) = x_1$$

Квартира	x_1 Общая, м²	x_2 Жилая, м²	A	B
1	60	40	60	60
2	75	50	75	75

При $x_2 = \frac{2}{3}x_1$ обе модели дают один и тот же прогноз.

Новое соотношение площадей — разные прогнозы

$$f_A(x) = 3x_1 - 3x_2, \qquad f_B(x) = x_1$$

Квартира	x_1 Общая, м²	x_2 Жилая, м²	A	B
1	60	40	60	60
2	75	50	75	75
Новая	60	50	30	60

$$f_A(60, 50) = 180 - 150 = 30 \qquad f_B(60, 50) = 60$$

Общая площадь осталась 60 м², жилая выросла с 40 до 50 м². Прогноз A снизился с 60 до 30, прогноз B остался 60.

Регуляризация: ограничиваем выбор решения

Регуляризация — управление обучением с помощью ограничений на сложность модели или штрафов в целевой функции.

Контроль переобучения

Сдерживать подстройку к случайным особенностям обучающих данных.

Устойчивость решения

Сдерживать большие компенсирующие коэффициенты и их реакцию на изменение данных.

В линейных моделях будем ограничивать величину коэффициентов при сопоставимых масштабах признаков.

Слишком сильное ограничение может ухудшить прогнозы. Его пользу проверяем по отложенным данным.

Исходная задача — метод наименьших квадратов

$$f_{\beta_0,\beta}(x) = \beta_0 + x^\top \beta$$

β_0 — свободный член; $\beta = (\beta_1, \ldots, \beta_p)^\top$ — коэффициенты при признаках.

$$R_n(\beta_0, \beta) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \beta_0 - x_i^\top \beta)^2$$

$$(\hat{\beta}_0, \hat{\beta}) \in \arg\min_{\beta_0,\beta} R_n(\beta_0, \beta)$$

МНК выбирает минимальную MSE; R_n — эмпирический риск.

При одинаковых столбцах МНК видит только сумму

$$x_2 = x_1, \quad \beta_0 = 0 \quad \Rightarrow \quad f(x) = (\beta_1 + \beta_2)x_1$$

β_1	β_2	Сумма
3	-2	1
1	0	1
0.5	0.5	1

Если сумма коэффициентов равна 1, прогнозы и MSE одинаковы на всех таких обучающих объектах.

Если один набор оптимален по МНК, то оптимальны и все наборы с той же суммой. Коэффициенты определяются неоднозначно.

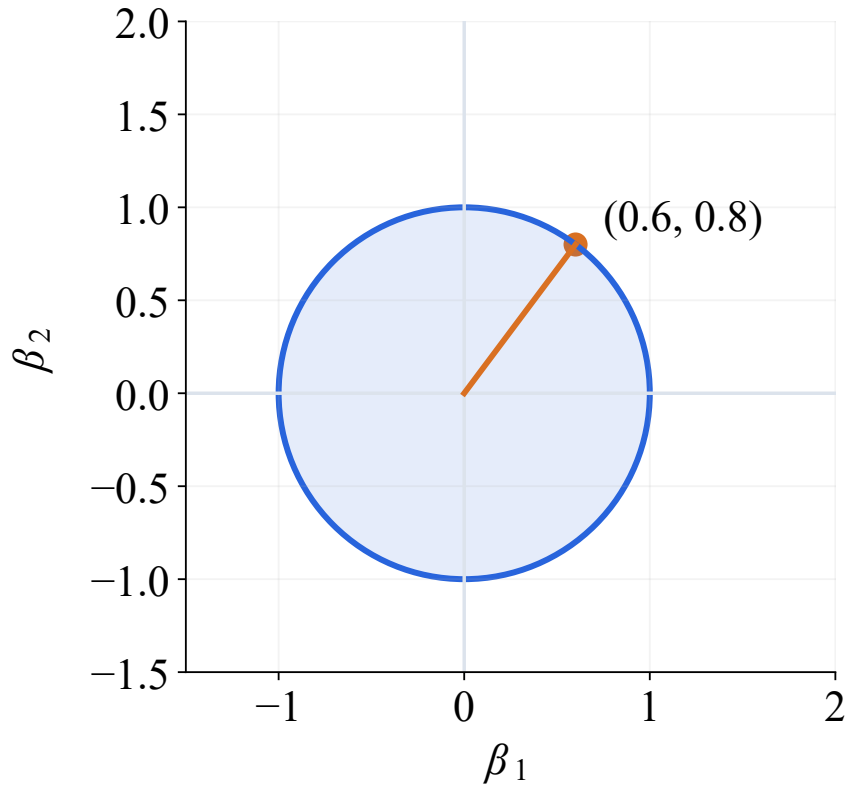
Добавим требование к величине коэффициентов

β_1	β_2	Сумма	$\|\|\beta\|\|_2^2$
3	-2	1	13
1	0	1	1
0.5	0.5	1	0.5

Одинаковая подгонка допускает и большие компенсирующие коэффициенты, и более умеренные.

Идея Ridge: учитывать величину всего вектора коэффициентов при выборе решения.

L2-норма — длина вектора коэффициентов



$$\|\beta\|_2 = \sqrt{\sum_{j=1}^{p} \beta_j^2}$$

$$\|\beta\|_2^2 = \sum_{j=1}^{p} \beta_j^2$$

$$\|\beta\|_2^2 \leq t$$

В двух координатах — диск радиуса $\sqrt{t}$.

Ridge: минимизируем MSE в допустимой области

$$\min_{\beta_0,\beta} R_n(\beta_0, \beta)$$

$$\text{при } \|\beta\|_2^2 \leq t, \qquad t \geq 0$$

Меньшее t оставляет более узкую область допустимых коэффициентов. Ищем лучшую подгонку внутри неё.

$t = 1:$ $(3, -2)$ недопустимы: квадрат нормы 13. $(1, 0),\ (0.5, 0.5)$ допустимы: квадрат нормы 1 и 0,5.

Если решение МНК лежит внутри области, ограничение сохраняет его.

Та же идея через штраф: цена больших коэффициентов

$$J_\lambda^{\text{Ridge}}(\beta_0, \beta) = R_n(\beta_0, \beta) + \lambda \|\beta\|_2^2$$

Первое слагаемое — ошибка подгонки.

Второе — цена величины коэффициентов.

$$\lambda \geq 0, \qquad \lambda = 0 \;\Rightarrow\; \text{МНК}$$

При большем λ уменьшение нормы получает больший вес относительно уменьшения MSE.

Регуляризация добавляет ограничение или штраф к критерию обучения. Свободный член β_0 не штрафуем; при согласованных t и λ две задачи имеют общее решение.

При равной MSE штраф различает решения

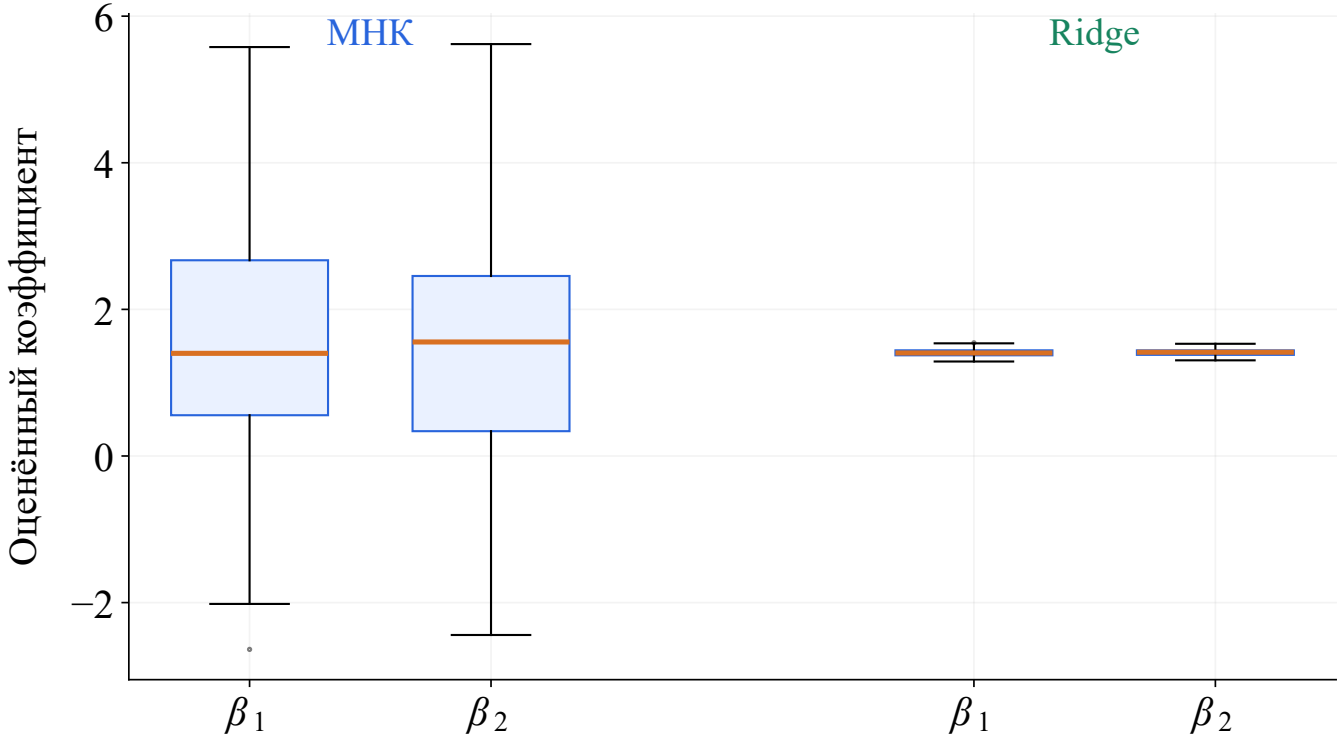
$$R_n = 2, \qquad \lambda = 0.2, \qquad J_\lambda = R_n + \lambda \|\beta\|_2^2$$

β_1	β_2	Сумма	$\|\beta\|_2^2$	Штраф	$J_{0.2}$
3	-2	1	13	2,6	4,6
1	0	1	1	0,2	2,2
0.5	0.5	1	0.5	0,1	2,1

Из трёх кандидатов меньший критерий у (0,5; 0,5).

Во всей задаче Ridge сопоставляет оба слагаемых: возможен обмен небольшой прибавки MSE на существенное уменьшение нормы.

Ridge уменьшает разброс оценённых коэффициентов



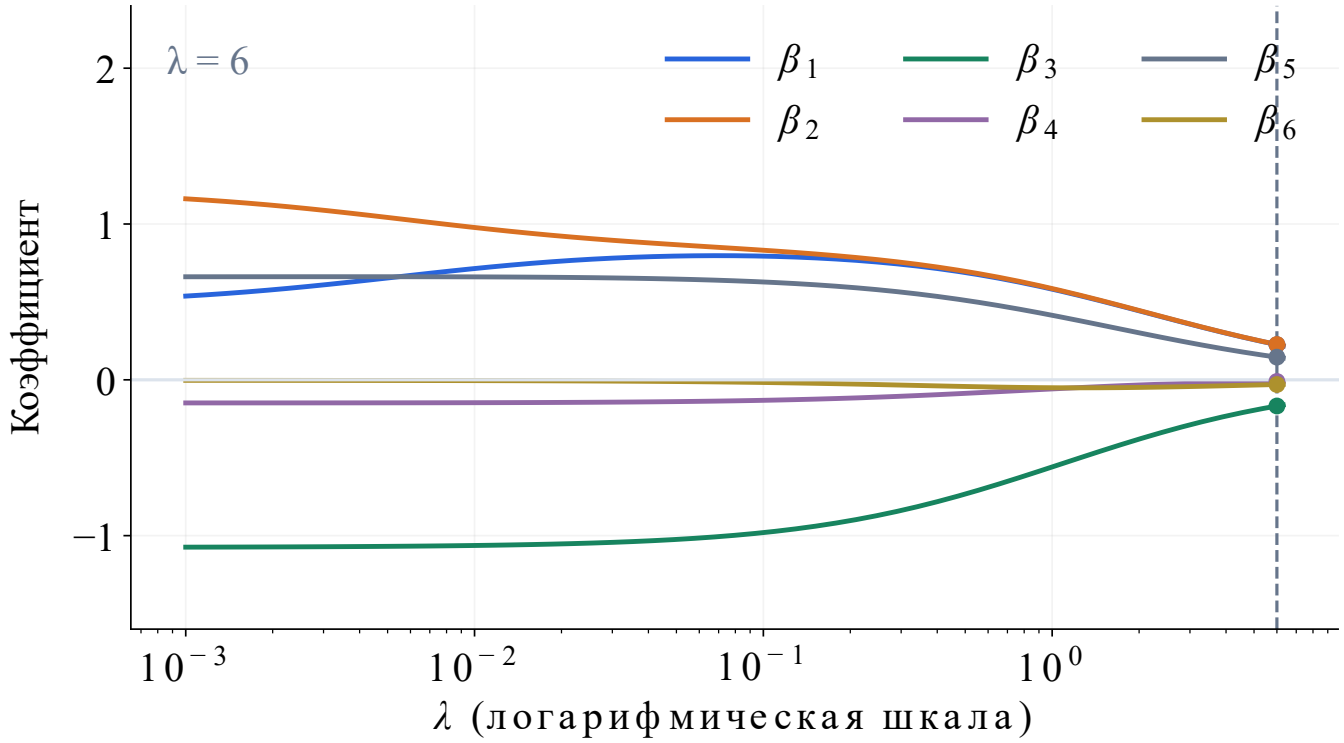
Один прямоугольник — центральные 50% из 100 оценок соответствующего коэффициента; линия внутри — медиана.

У МНК большие положительные и отрицательные вклады компенсируют друг друга.

При выбранном штрафе Ridge оценки заметно менее изменчивы.

Качество на новых объектах проверяем отдельной валидацией.

Траектории Ridge: одна выборка, разные λ



Признаки 1–2 сильно коррелируют; 3 и 5 информативны; 4 и 6 — шумовые.

Для каждого λ обучаем отдельную модель на тех же данных.

130 объектов; 6 стандартизированных признаков. Каждая линия — один $\hat{\beta}_j(\lambda)$.

Общий квадрат нормы не растёт. Отдельные коэффициенты могут перераспределять вклад.

При сильном штрафе наклоны стремятся к нулю, а прогнозы — к среднему y.

Одни прогнозы — разные единицы и штрафы

Площадь в м²

$$x = 60, \quad \beta = 20$$

$$\beta x = 1200$$

$$\beta^2 = 400$$

Та же площадь в см²

$$x' = 600\,000, \quad \beta' = 0.002$$

$$\beta' x' = 1200$$

$$(\beta')^2 = 0.000004$$

Прогноз сохранился, цена этого коэффициента в L2-штрафе изменилась в 10^8 раз.

$$x_{ij}^{\text{std}} = \frac{x_{ij} - \bar{x}_j}{s_j}$$

Перед регуляризацией приводим признаки к сопоставимому масштабу. Средние и s считаем по обучающим данным.

От Ridge к Lasso: ограничим сумму модулей

$$\textbf{Ridge: } \|\beta\|_2^2 = \sum_j \beta_j^2$$

$$\textbf{Lasso: } \|\beta\|_1 = \sum_j |\beta_j|, \qquad \|(3,-2)\|_1 = 5$$

Ограниченная задача

$$\min_{\beta_0,\beta} R_n \quad \text{при } \|\beta\|_1 \le t$$

Штрафная задача

$$J_\lambda^{\text{Lasso}} = R_n + \lambda\|\beta\|_1$$

Lasso сдерживает величину коэффициентов с помощью L1. Формула линейного прогноза остаётся прежней; свободный член не штрафуется.

L1 может давать разреженное решение

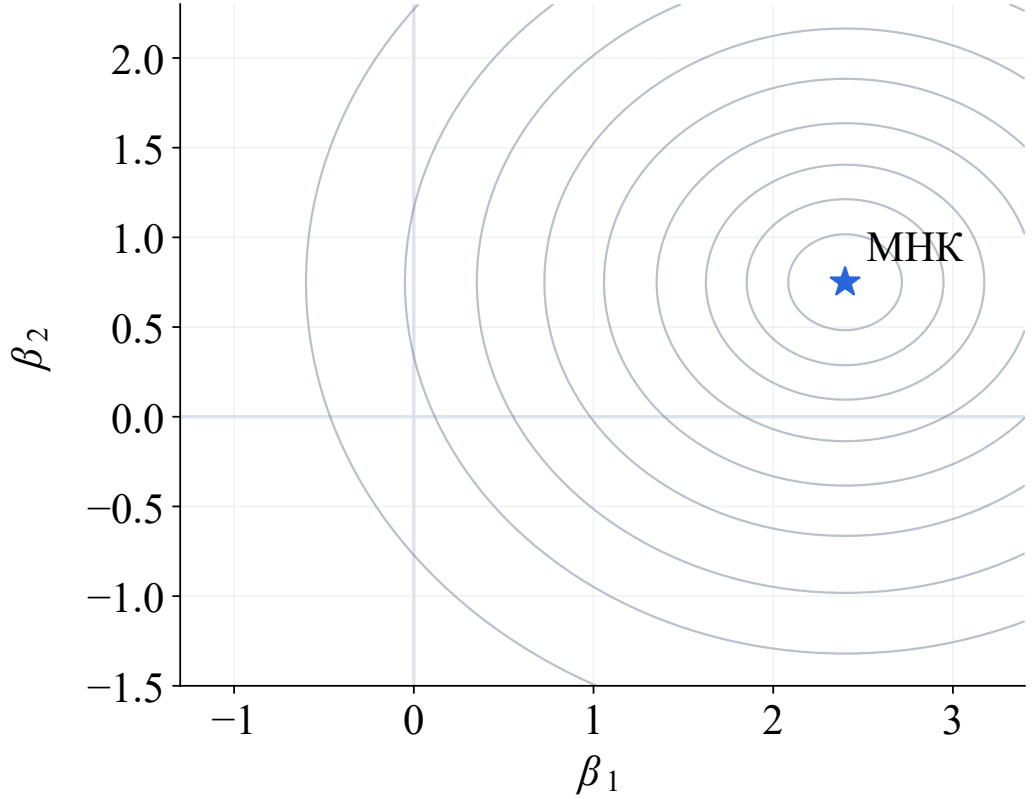
L1-штраф имеет изломы при нулевых коэффициентах. Минимум может оказаться в одной из таких точек — механизм увидим на геометрии.

$$f(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3$$

$$\hat{\beta}_2 = 0 \quad \Rightarrow \quad \hat{f}(x) = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_3 x_3$$

Нулевой коэффициент исключает признак из правила прогноза. Если многие коэффициенты равны нулю, решение называют разреженным.

Плоскость коэффициентов и линии уровня

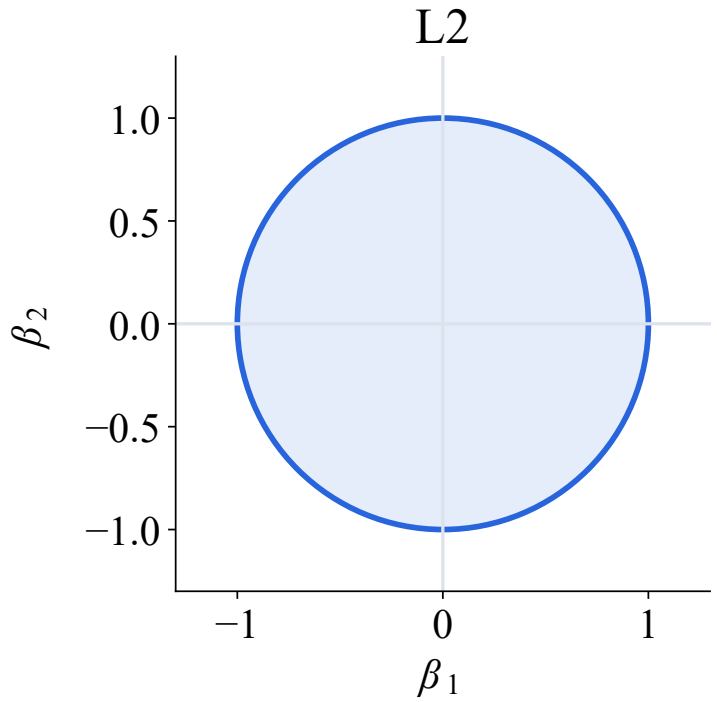


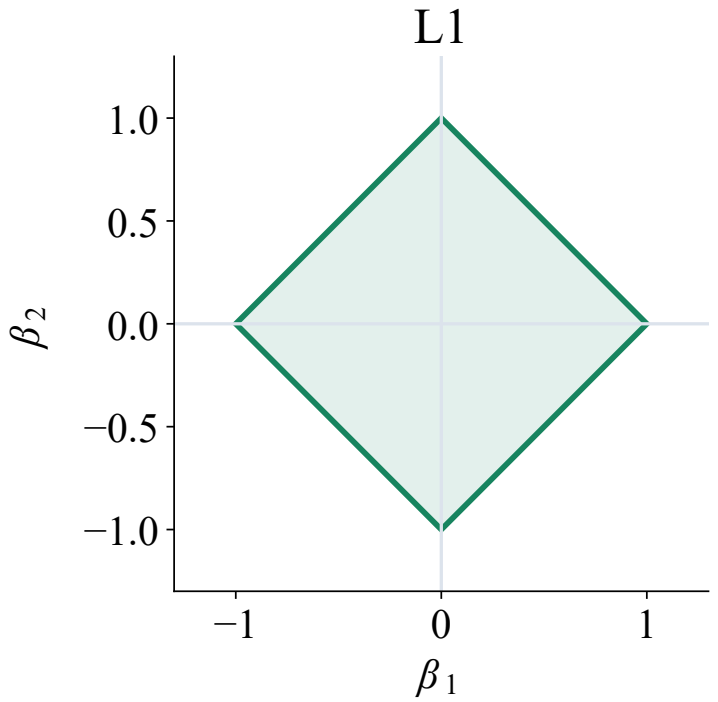
Каждая точка (β_1, β_2) — одна модель.

На одной линии уровня — одинаковая MSE.

Если решение МНК единственно, линии уровня MSE в пространстве двух коэффициентов образуют вложенные эллипсы. В центре — минимум МНК.

Круг L2 и ромб L1



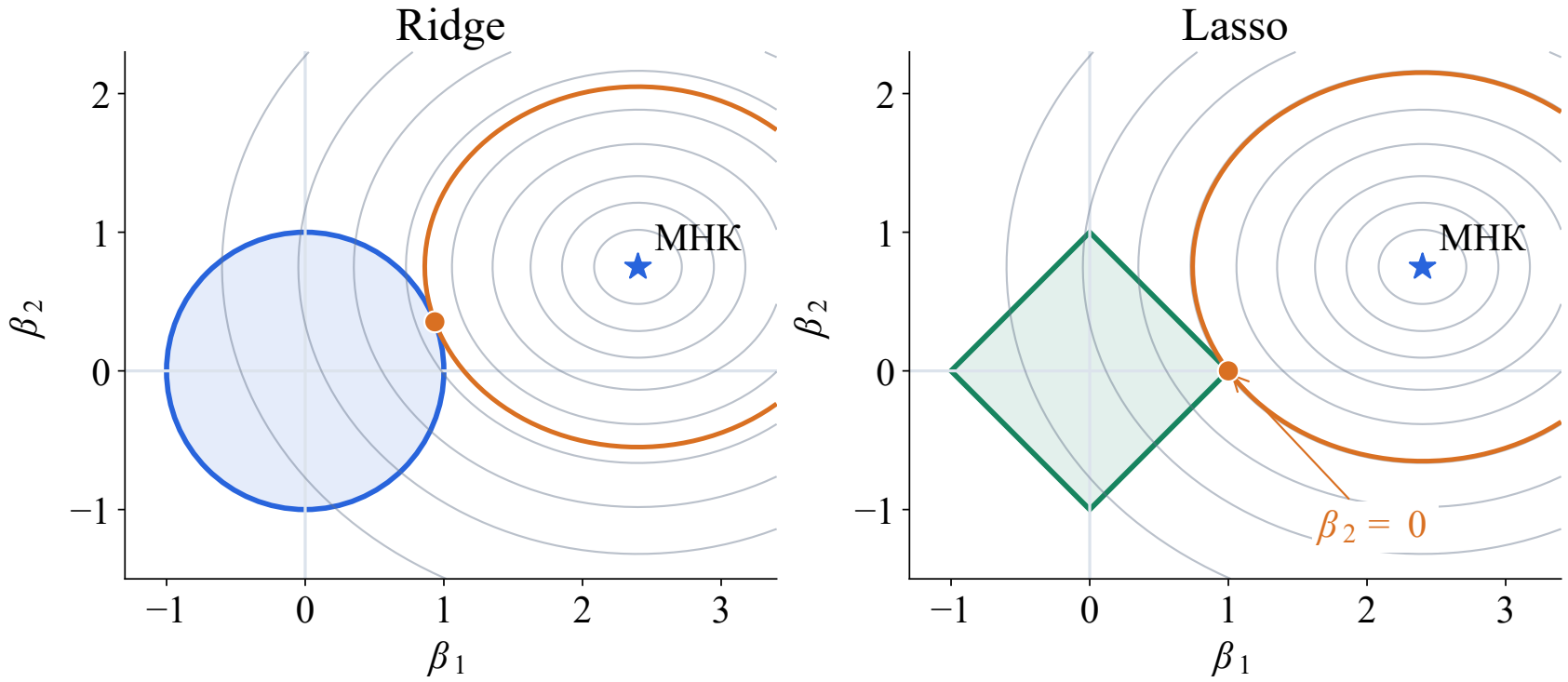


$$\beta_1^2 + \beta_2^2 \leq t$$

$$|\beta_1| + |\beta_2| \leq t$$

Области допустимых коэффициентов; на рисунке t = 1.

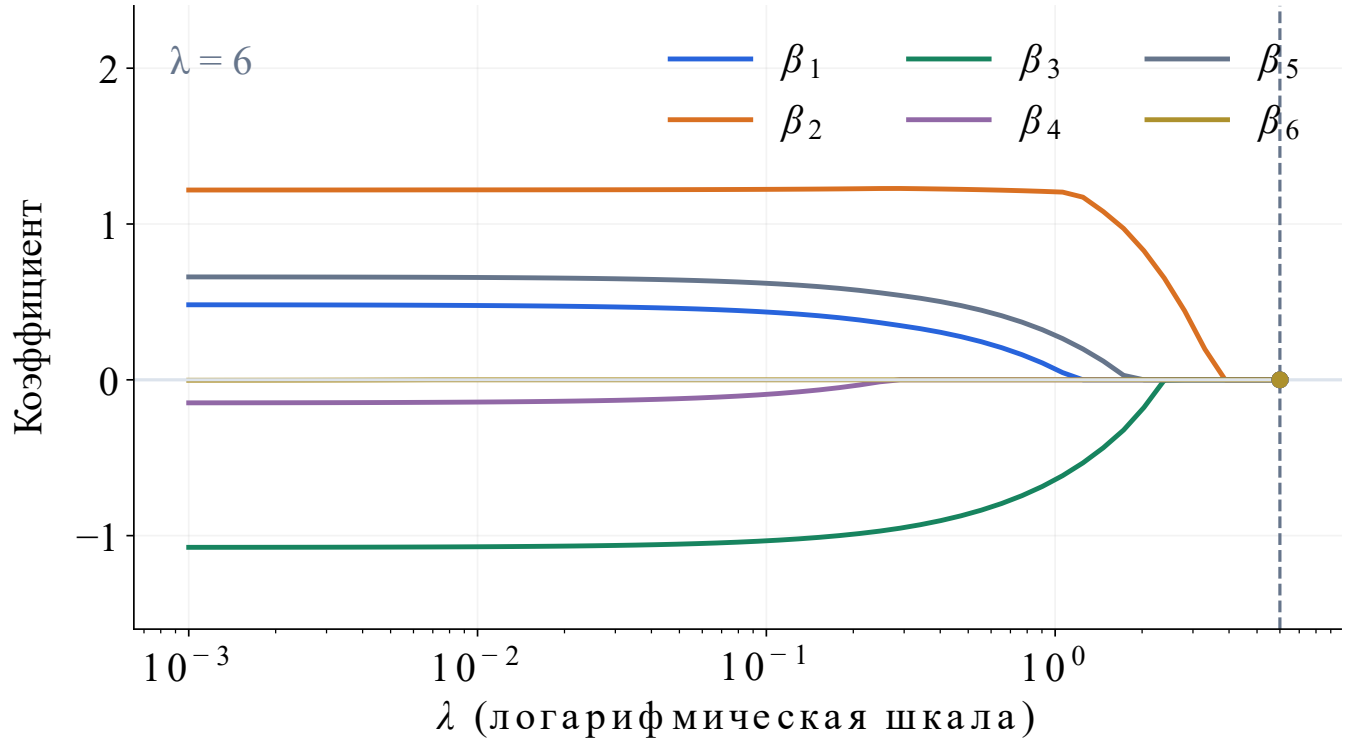
Как ограничение сдвигает минимум



Касание вершины: второй коэффициент равен нулю.

Расширяем линии уровня от центра до первого касания. Для Lasso касание вершины возможно, но не обязательно.

Траектории Lasso: координаты доходят до нуля



Признаки 1–2 сильно коррелируют; 3 и 5 информативны; 4 и 6 — шумовые.

Эксперимент: 130 объектов, 6 стандартизированных признаков.

Ненулевых коэффициентов: 0

Lasso подбирает коэффициенты совместно: снижение ошибки сопоставляется с L1-штрафом.

При сильном штрафе все наклоны нулевые — остаётся постоянный прогноз.

Elastic Net сочетает L1 и L2

$$J_{\lambda,\rho} = R_n + \lambda \left[\rho \|\beta\|_1 + (1-\rho)\|\beta\|_2^2\right]$$

$$\lambda \geq 0, \qquad 0 \leq \rho \leq 1$$

$$\rho = 1 : \ \text{Lasso} \qquad\qquad \rho = 0 : \ \text{Ridge с весом } \lambda$$

L1 сохраняет возможность отбора признаков; L2 помогает распределять вклад между связанными признаками.

λ задаёт общую силу регуляризации, ρ — соотношение двух штрафов.

K log loss тоже можно добавить штраф

$$p_{\beta_0,\beta}(x) = \sigma(\beta_0 + x^\top \beta)$$

$$R_n^{\log} = \frac{1}{n}\sum_i L(y_i, p_i)$$

$$J_\lambda^{\log} = R_n^{\log} + \lambda\|\beta\|_2^2$$

L1 или L2 ограничивают коэффициенты логистической регрессии. Это может стабилизировать решение и сдерживать подгонку к шуму.

В sklearn меньше C — сильнее регуляризация. Свободный член не штрафуем; численное соответствие C и λ зависит от нормировки.

Что меняет регуляризация

- Ridge сдерживает L2-норму; Lasso может занулять коэффициенты; Elastic Net сочетает L1 и L2.
- Масштаб признаков влияет на штраф. Свободный член не штрафуем.
- Сила ограничения — настройка модели. Меньшая train-ошибка не гарантирует лучший прогноз.

Как выбрать силу штрафа и проверить, помогает ли он новым прогнозам?

Вспомним train/test: обучение и независимая проверка

Разделяем данные до обучения, чтобы проверить прогнозы на объектах, которые модель ещё не видела.



Train — обучение

По этим объектам находим параметры модели.

Test — финальная проверка

Сравниваем прогнозы готовой модели с фактическими ответами.

Для сравнения вариантов внутри разработки используем train/validation. Финальный test оставляем отдельно от выбора.

Может ли оценка на отложенных объектах измениться, если по-другому разделить те же данные?

Одно разбиение: обучаем и оцениваем

30 объектов из синтетической выборки. Фиксированная процедура: стандартизация по train → Ridge с λ = 1 → MSE на validation.

Train: 24 объекта Validation: 6 объектов



24 train
Обучающие объекты

→

Модель 1
Обучаем заново

→

MSE = 2,235
На 6 validation-объектах

Validation: № 1, 6, 12, 19, 22, 24

Что изменится, если выбрать другие train/validation-объекты?

Изменили состав частей — обучаем новую модель

Те же 30 объектов, та же процедура и λ = 1. Изменилось только разбиение; часть объектов поменяла роль.

Train: 24 объекта Validation: 6 объектов Обводка — объект сменил роль



24 train
Обучающие объекты

→

Модель 2
Обучаем заново

→

MSE = 0,694
На 6 validation-объектах

Validation: № 5, 10, 11, 14, 22, 24

Оценка зависит и от обучающих объектов, и от состава отложенной части.

Кросс-валидация повторяет обучение и проверку

Кросс-валидация (CV) оценивает процедуру обучения на нескольких разбиениях данных.

Текущие train и validation → Заново обучаем на train → Оцениваем на validation

Повторяем цикл на других разбиениях и объединяем полученные оценки.

Пока фиксируем модель, настройки, преобразования и метрику. Каждый проход даёт отдельную обученную модель.

K-fold: от исходных данных к фолдам внутри dev

100 исходных объектов

80 — данные для разработки (dev)

F_1	F_2	F_3	F_4	F_5
16 объектов	16 объектов	16 объектов	16 объектов	16 объектов

20 — test

Откладываем для финальной проверки

K-fold: делим dev на K непересекающихся частей — фолдов.

Из 100 объектов 20 отложены для финального test, остальные 80 — для разработки и CV. При K = 5 каждый dev-объект проверяем ровно один раз.

Другой validation-фолд — другая обученная модель

Train Validation

Финальный test — вне CV

Проход	F_1	F_2	F_3	F_4	F_5	Модель	Ошибка
2	Train	Valid	Train	Train	Train	Модель 2	$\mathrm{MSE}_2 = 4{,}7$

Обучаем новую модель на 64 объектах → проверяем на 16 других.

На прошлом проходе: Модель 1, validation — первый фолд, MSE = 4,1.

Когда F_2 становится validation, F_1 возвращается в train. Обучающая часть меняется — коэффициенты находим заново.

Пять моделей, пять ошибок, одно среднее

Train Validation

Финальный test — вне CV

Проход	F_1	F_2	F_3	F_4	F_5	Модель	Ошибка
1	Valid	Train	Train	Train	Train	Модель 1	$\mathrm{MSE}_1 = 4{,}1$
2	Train	Valid	Train	Train	Train	Модель 2	$\mathrm{MSE}_2 = 4{,}7$
3	Train	Train	Valid	Train	Train	Модель 3	$\mathrm{MSE}_3 = 3{,}9$
4	Train	Train	Train	Valid	Train	Модель 4	$\mathrm{MSE}_4 = 5{,}8$
5	Train	Train	Train	Train	Valid	Модель 5	$\mathrm{MSE}_5 = 4{,}5$

$$\text{CV-MSE} = \frac{4{,}1 + 4{,}7 + 3{,}9 + 5{,}8 + 4{,}5}{5} = 4{,}6$$

CV-оценка относится к процедуре обучения

Пять проходов CV

Каждая модель: 64 train → 16 validation.

$$\text{CV-MSE} = 4.6$$

Средняя отложенная ошибка пяти моделей при этих разбиениях.

Одна итоговая модель

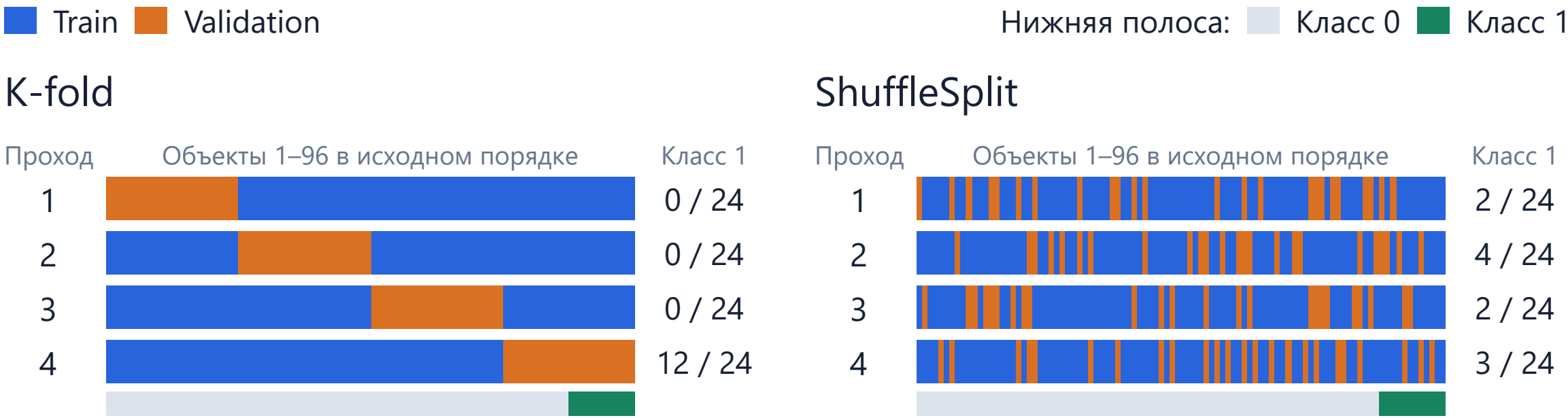
Заново обучим на всех 80 dev-объектах.

Её параметры и размер обучения отличаются от CV-моделей. Для проверки выбранного решения сохранён test.

Мы оценили работу заданного способа обучения на 64 объектах, а не одну окончательную модель на 80.

Обучающие части пересекаются; пять ошибок не являются независимыми экспериментами.

K-fold и ShuffleSplit: как меняется состав проверки



Четыре непересекающихся фолда. В исходном порядке классы сгруппированы: все положительные попали в последний фолд.

Четыре случайных разбиения заново. Один объект может проверяться несколько раз, другой — ни разу.

96 объектов; на каждом из четырёх проходов 24 объекта используются для валидации.

Стратификация и группы отвечают разным задачам

Train Validation

Нижняя полоса: Класс 0 Класс 1

Stratified K-fold



В каждом фолде 3 положительных и 21 отрицательный объект: сохраняем доли классов.

Group K-fold



12 пациентов по 8 записей. Проверяем на новых пациентах: все записи выбранного пациента целиком в validation.

Если объектов класса меньше числа фолдов, стратификация не обеспечит его присутствие в каждом validation-фолде. Для временных данных учитываем порядок наблюдений.

Параметры и гиперпараметры

$$\hat{\beta}_0,\ \hat{\beta}$$

Параметры

Находятся по данным внутри конкретного обучения.

$$\lambda,\ C,\ k$$

Гиперпараметры

Задаются до обучения; сравниваем процедуры с разными настройками.

Почему минимальная train-MSE не выбирает полезный штраф

МНК уже минимизирует MSE на данной обучающей выборке.

$$\min_{\beta_0,\beta} R_n \le R_n(\hat{\beta}_{0,\lambda}, \hat{\beta}_\lambda)$$

Выбор по train-MSE предпочитает отсутствие регуляризации.

Для новых прогнозов сравниваем ошибку на отложенных объектах.

Учебный пример одной validation-части

λ	MSE на train	MSE на validation
0	1,2	3,8
0,1	1,4	2,6
1	1,9	2,2
10	3,1	3,4

CV-подбор Ridge: один набор данных и общие фолды

Рассмотрим новый эксперимент: 150 объектов для разработки, 60 взаимосвязанных признаков и зашумлённая целевая переменная.

$$\Lambda = \{0.0001,\ 0.001,\ 0.01,\ 0.1,\ 1,\ 10,\ 100\}$$

F_1	F_2	F_3	F_4	F_5
30 объектов	30 объектов	30 объектов	30 объектов	30 объектов

Для каждого λ: пять новых моделей, 120 train / 30 validation на проходе. Для всех кандидатов используем одни и те же фолды.

Средние и стандартные отклонения признаков оцениваем отдельно на train каждого прохода.

Первое λ: пять обучений и пять ошибок

$\lambda = 0.0001$

Вычисляем среднее пяти ошибок

F_1 30 объектов	F_2 30 объектов	F_3 30 объектов	F_4 30 объектов	F_5 30 объектов

Пять отдельных моделей → пять отложенных ошибок → одна CV-MSE

λ	F_1	F_2	F_3	F_4	F_5	Среднее
0,0001	3,425	6,159	3,282	3,817	4,072	4,151
0,001	—	—	—	—	—	—

$$\text{CV-MSE}(\lambda) = \frac{\text{MSE}_1 + \cdots + \text{MSE}_5}{5}$$

Усредняем пять ошибок внутри одной строки.

Второе λ: повторяем весь CV-цикл

$\lambda = 0.001$

Вычисляем среднее пяти ошибок

F_1 30 объектов	F_2 30 объектов	F_3 30 объектов	F_4 30 объектов	F_5 30 объектов

Пять отдельных моделей → пять отложенных ошибок → одна CV-MSE

λ	F_1	F_2	F_3	F_4	F_5	Среднее
0,0001	3,425	6,159	3,282	3,817	4,072	4,151
0,001	3,333	5,978	3,178	3,698	3,972	4,032

$$\text{CV-MSE}(\lambda) = \frac{\text{MSE}_1 + \cdots + \text{MSE}_5}{5}$$

Усредняем пять ошибок внутри одной строки.

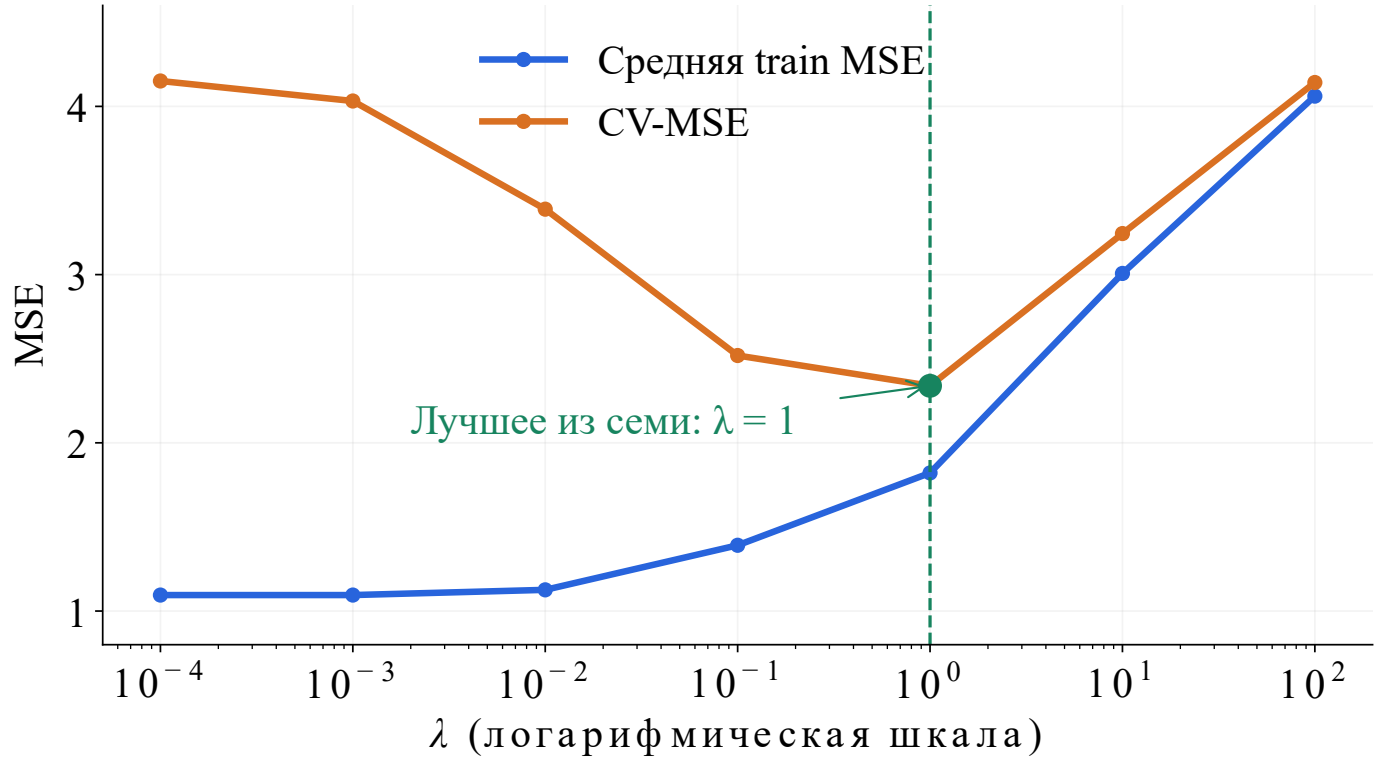
Полная таблица: сравниваем средние семи кандидатов

λ	F_1	F_2	F_3	F_4	F_5	CV-MSE
0,0001	3,425	6,159	3,282	3,817	4,072	**4,151**
0,001	3,333	5,978	3,178	3,698	3,972	**4,032**
0,01	2,768	5,054	2,602	3,071	3,448	**3,389**
0,1	1,891	3,815	2,022	2,241	2,626	**2,519**
1	2,033	3,450	2,141	1,884	2,185	**2,339**
10	3,503	4,441	3,249	2,466	2,562	**3,244**
100	4,434	5,387	4,225	3,329	3,336	**4,142**

Каждая ячейка — валидационная ошибка. Фолды в столбцах одинаковы для всех λ; среднее в строке — CV-MSE.

Наименьшая средняя ошибка: λ = 1, CV-MSE = 2,339.

Кривая CV — те же средние из таблицы



λ = 1 — лучший из семи проверенных кандидатов на этих данных.

35 обучений для выбора — и новое обучение после него

Выбор: 7 значений λ × 5 фолдов

35 обучений

Сравниваем CV-MSE.
Выбираем λ = 1.

Окончательное обучение — refit

1 новая модель

Обучаем стандартизатор и Ridge при $\lambda = 1$ на всех 150 dev.

Пять CV-моделей выбранного λ дали среднюю ошибку 2,339. Их коэффициенты не объединяем: итоговую модель обучаем заново.

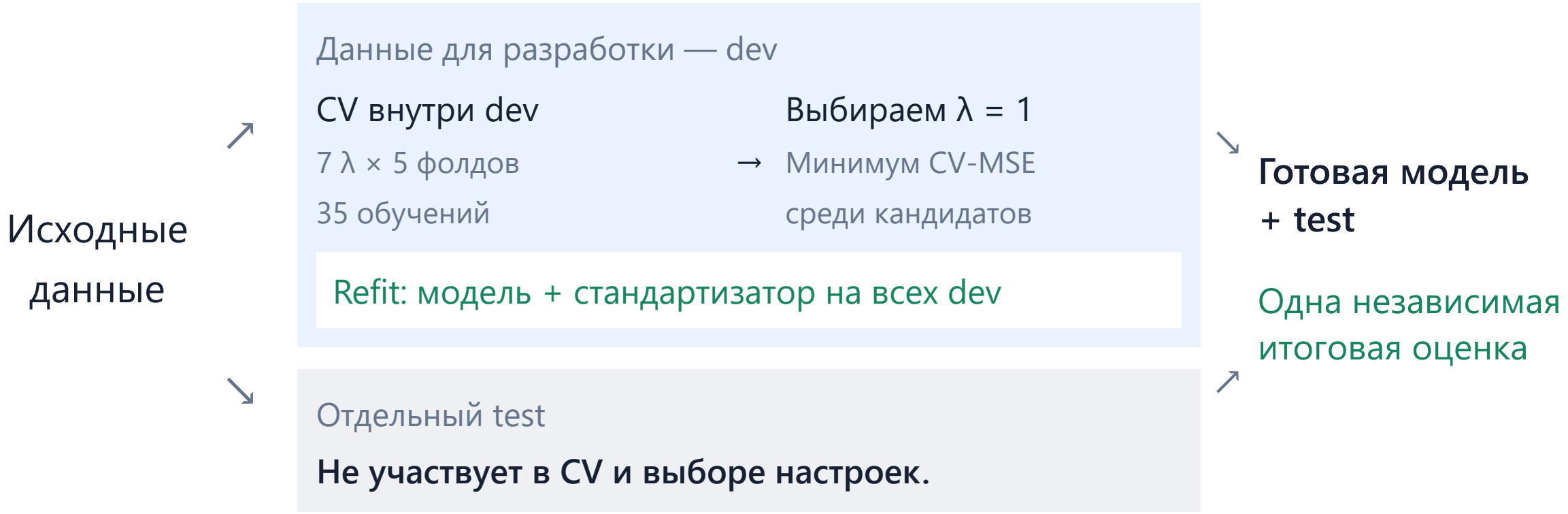
Стандартизацию обучаем внутри каждого прохода

Порядок для каждого CV-прохода

1. На 120 train оцениваем средние и стандартные отклонения.
2. По этим статистикам преобразуем train и 30 validation.
3. Обучаем Ridge на преобразованном train.
4. Прогнозируем для validation и считаем MSE.

При refit и статистики стандартизации, и Ridge обучаем на всех dev. Test только преобразуем и используем для итоговой оценки.

Финальный протокол: dev и test идут разными ветвями



По CV выбирали λ, поэтому лучшая CV-оценка может быть оптимистичной. Отдельный test проверяет зафиксированное решение.

CV сравнивает процедуры обучения

- В каждом проходе заново обучаем всю процедуру на train и проверяем на validation.
- Настройки сравниваем на одинаковых фолдах по средней метрике.
- После выбора — refit на всех dev. Отдельный test проверяет зафиксированное решение.

Теперь можем сравнивать разные способы строить прогноз: общую формулу и локальное соседство.

Что у нас уже есть

Линейная модель

Коэффициенты задают общую зависимость.

Логистическая регрессия

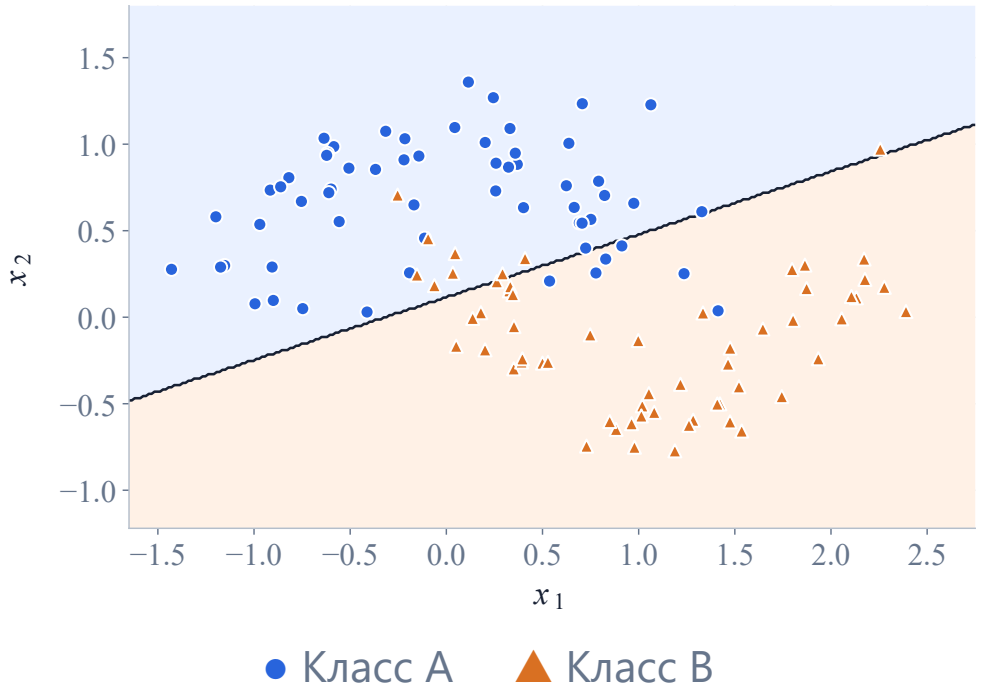
Score превращается в вероятность.

Кросс-валидация

Проверяем качество настройки.

Можно ли делать прогноз, не подбирая одну глобальную формулу?

Линейная граница и сложная геометрия классов



В исходных признаках одной прямой недостаточно.

Преобразования признаков могут изменить ситуацию.

А если известны метки ближайших наблюдений?

Предсказать по одному ближайшему объекту

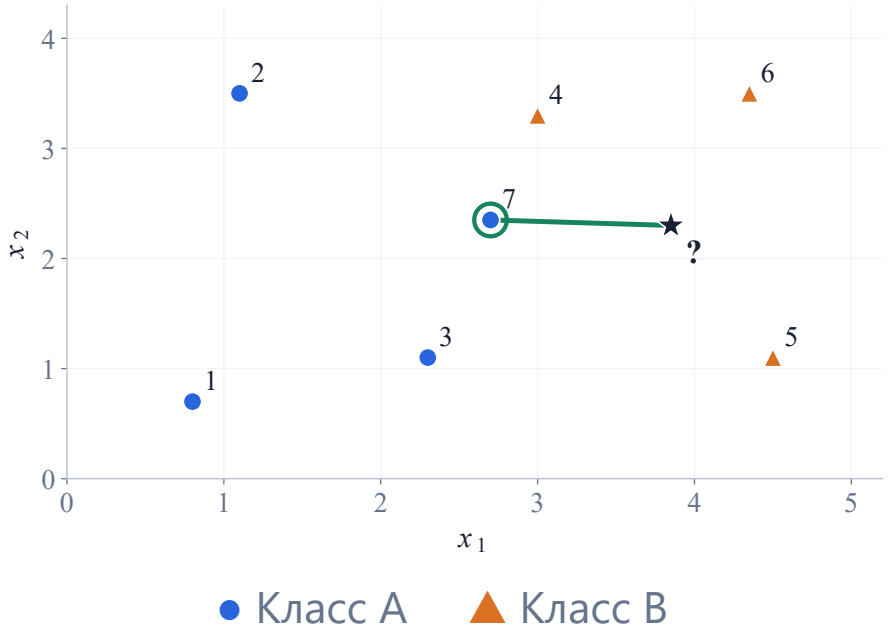


Обучающие точки имеют известные классы.

Какой класс назначить новой точке?

Ближайший — объект №7, класс A.
Прогноз новой точки — A.

Запишем 1NN математически



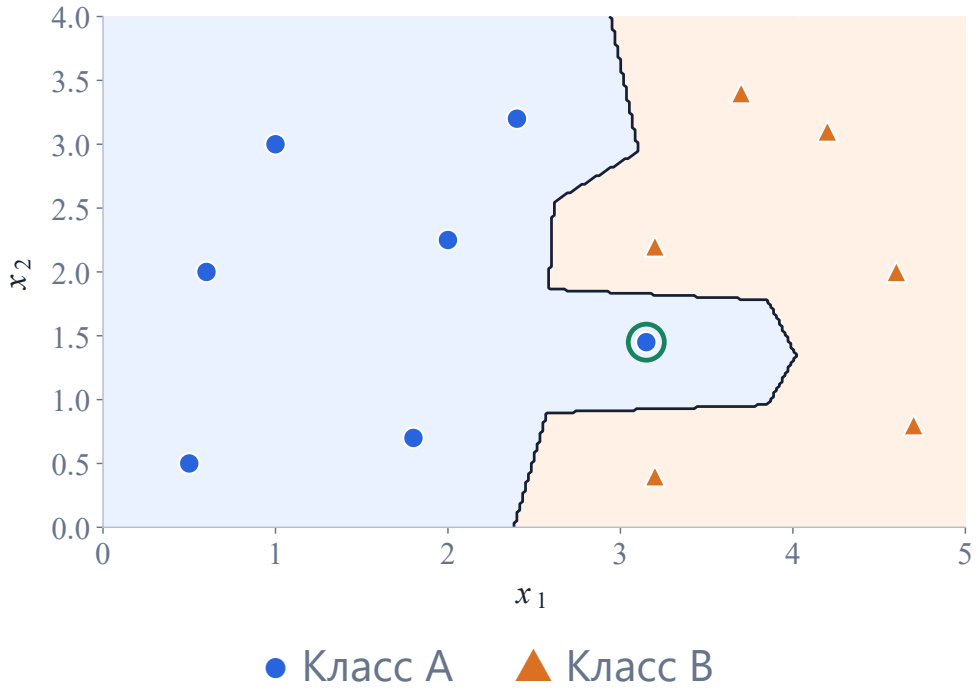
$$i^*(x) \in \arg\min_{i=1,\dots,n} d(x, x_i)$$

Ищем **индекс** ближайшего объекта.

$$\hat{y}(x) = y_{i^*(x)}$$

x — новый объект; i — номер обучающего; d — расстояние. Для другого объекта ближайший индекс может быть другим.

Цена избыточной локальности

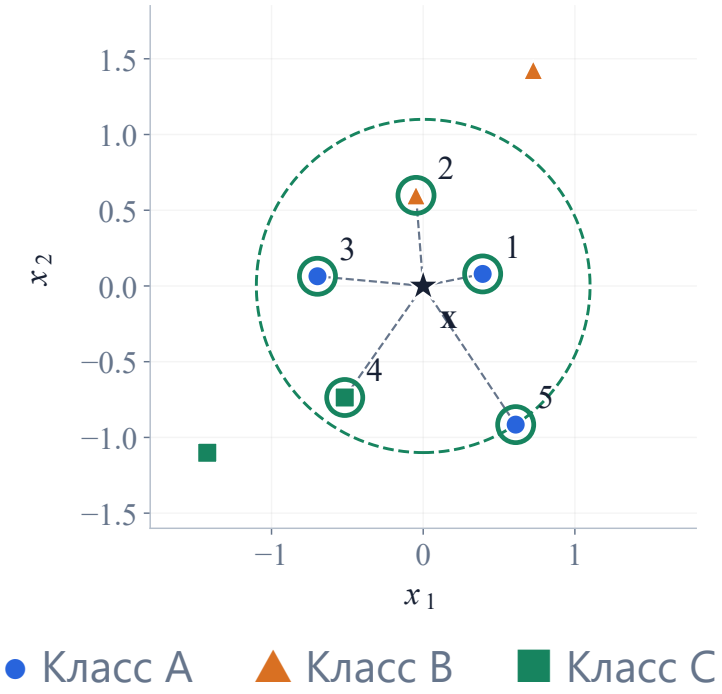


Одна обучающая точка получает собственную область решений.

Случайная ошибка разметки может изменить прогнозы вокруг неё.

Идеальная подстройка к train не гарантирует устойчивых новых прогнозов.

Не один, а k ближайших



$$k = 5$$

Каждый выбранный сосед голосует за свой класс.

Класс с наибольшим числом голосов становится прогнозом.

Правило голосования

$$\mathcal{N}_k(x) \text{ — индексы } k \text{ ближайших обучающих объектов}$$

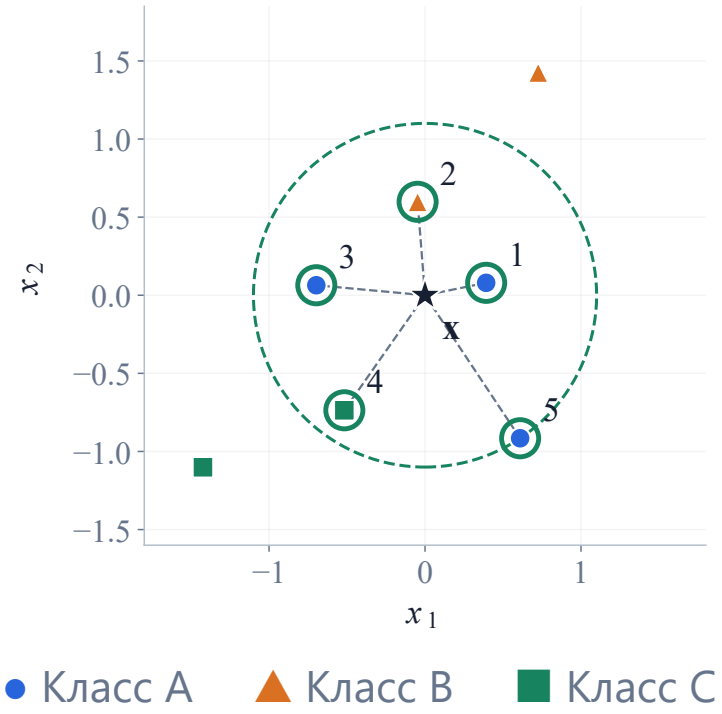
$$V_c(x) = \sum_{i \in \mathcal{N}_k(x)} \mathbb{I}(y_i = c)$$

Индикатор равен 1 для соседа класса c и 0 для остальных. Сумма — голоса за класс.

$$\hat{y}(x) \in \arg\max_{c \in \mathcal{Y}} V_c(x), \qquad \mathcal{Y} = \{1, \ldots, K\}$$

Argmax выбирает класс-победитель. Для ничьей нужно заранее заданное правило.

Одно голосование, три класса



Сосед	Расстояние	Класс
1	0.4	A
2	0.6	B
3	0.7	A
4	0.9	C
5	1.1	A

$$(V_A, V_B, V_C) = (3, 1, 1) \Rightarrow \hat{y} = A$$

Локальные доли классов

$$\hat{p}_c(x) = \frac{1}{k} \sum_{i \in \mathcal{N}_k(x)} \mathbb{I}(y_i = c)$$

$$(3, 1, 1) \xrightarrow{\div 5} (\hat{p}_A, \hat{p}_B, \hat{p}_C) = (0.6, 0.2, 0.2)$$

0.6 означает: 60% выбранных соседей относятся к A.

Доли классов — оценки условных вероятностей. Их качество проверяется на новых данных.

Что происходит при fit и predict

Логистическая регрессия

`fit`: оценить коэффициенты.

`predict`: применить общую функцию.

kNN

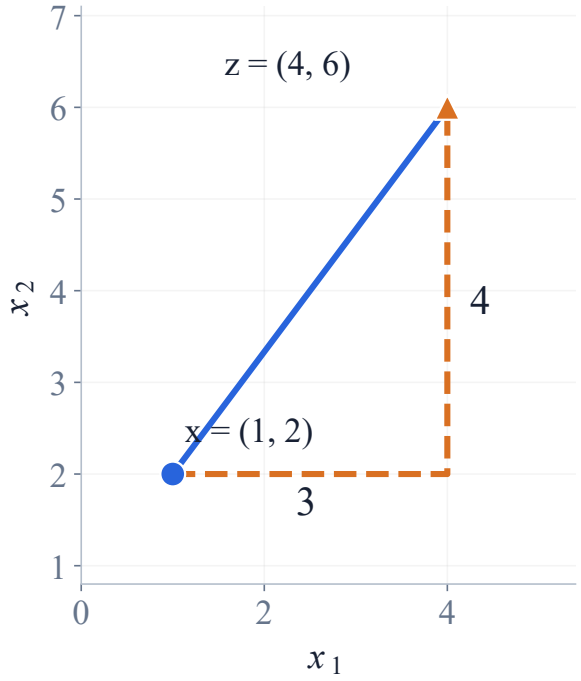
`fit`: сохранить train, подготовить поиск.

`predict`: расстояния → соседи → голоса.

Обучение по примерам: существенная часть работы приходится на прогноз.

Instance-based / lazy learning. Настройки kNN: число соседей, расстояние и веса.

Евклидово расстояние

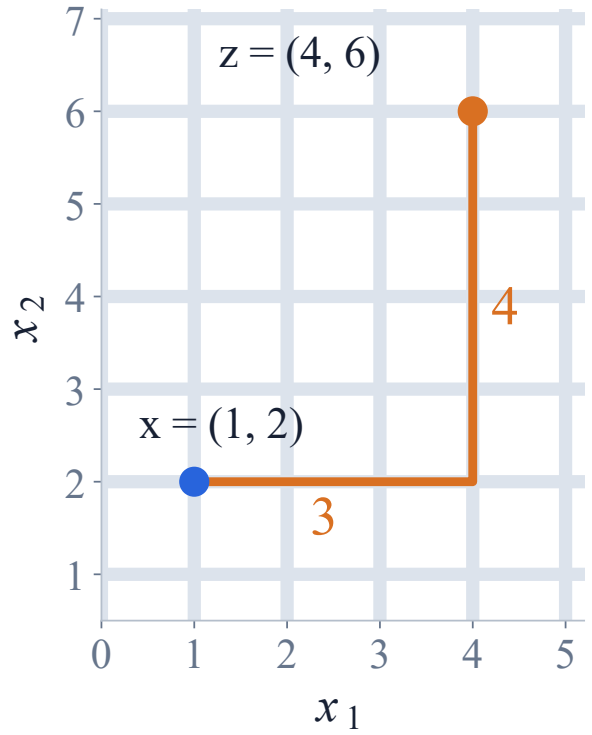


$$d_2(x, z) = \sqrt{\sum_{j=1}^{p}(x_j - z_j)^2}$$

$$d_2 = \sqrt{3^2 + 4^2} = 5$$

Привычная длина отрезка между точками.

Манхэттенское расстояние



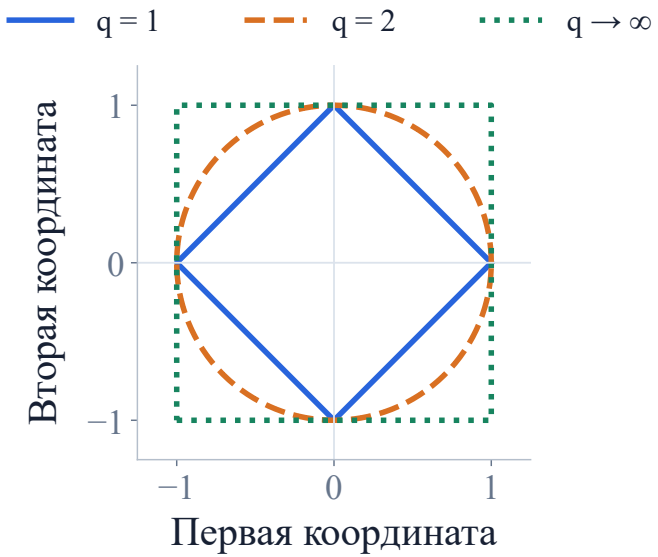
По прямоугольной сетке улиц идём вдоль кварталов, без движения наискосок.

$$d_1(x, z) = \sum_{j=1}^{p} |x_j - z_j|$$

$$d_1 = 3 + 4 = 7$$

Семейство расстояний Минковского

$$d_q(x, z) = \left(\sum_{j=1}^{p} |x_j - z_j|^q \right)^{1/q}, \qquad q \geq 1$$



q = 1: манхэттенское.

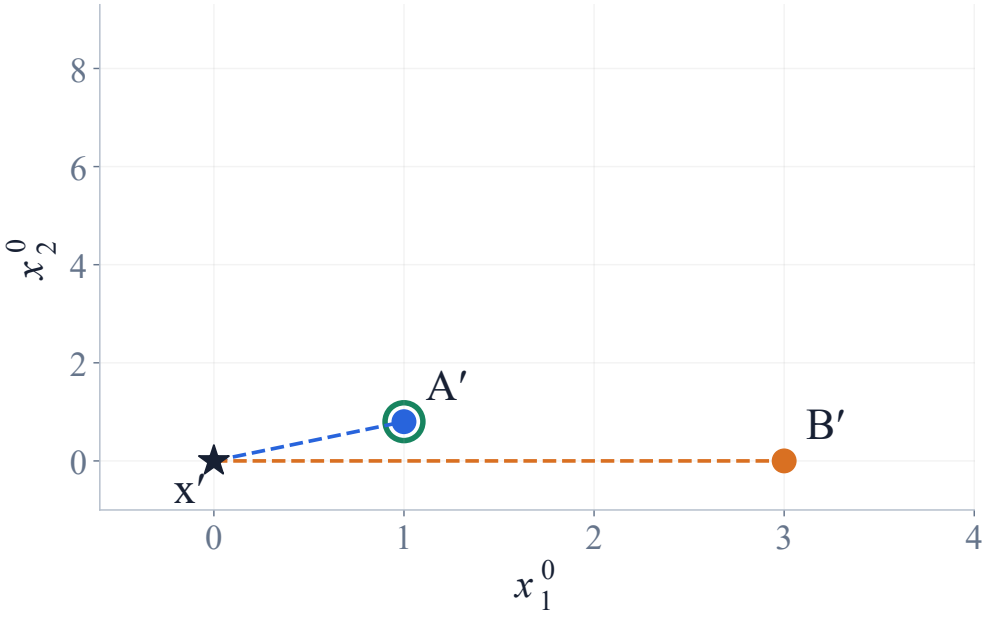
q = 2: евклидово.

q → ∞: расстояние Чебышёва.

$$d_\infty(x, z) = \max_j |x_j - z_j|$$

Определяется наибольшим различием координат.

Масштаб меняет ближайшего соседа



$$x' = (0, 0),\ A' = (1, 0.8),\ B' = (3, 0)$$

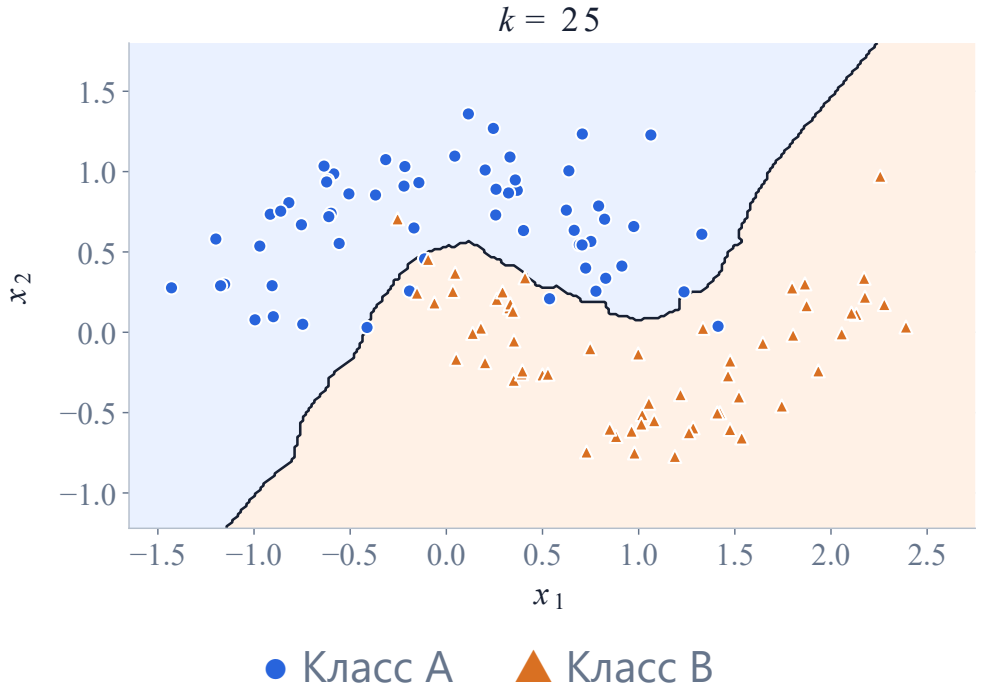
$$d(x', A') = \sqrt{1.64} \approx 1.28$$

$$d(x', B') = 3$$

Ближайший: A′.

Объекты те же. Смена единиц измерения может изменить прогноз kNN.

Что меняет число k?



Малое k: сильнее реакция на отдельные точки.

Больше k: сглаживаем локальное голосование.

Слишком большое k может скрыть полезную структуру.

Управляем локальностью и гибкостью. Качество не обязано меняться монотонно.

Выбрать k по валидации

$$k \in \{1, 3, 5, 7, 11, 15, 21\}$$

Те же CV-разбиения → Обучение и прогнозы → Сравнение метрики

Почему train accuracy подталкивает к k = 1?

Если сама train-точка включена в поиск, она обычно оказывается своим ближайшим соседом. Проверка на новых объектах снимает это преимущество.

Подготовка признаков обучается внутри train-фолда. Финальный test остаётся отдельно.

Близкие соседи должны ещё и найтись

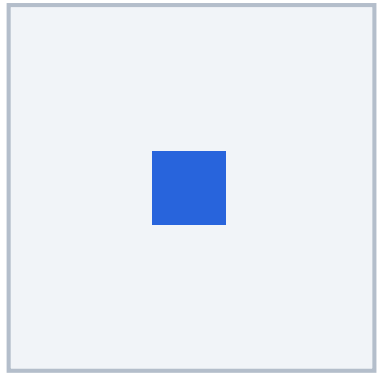
Хотим узкую окрестность и достаточно наблюдений в ней. При той же относительной стороне её доля пространства быстро уменьшается.

1 признак



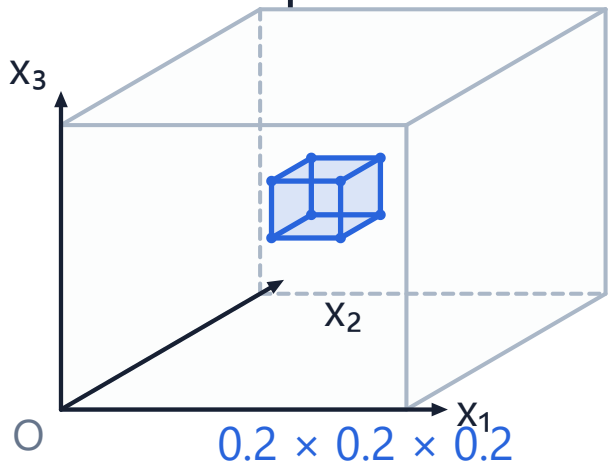
$r = 0.2 \to 20\%$

2 признака



$r^2 = 0.04 \to 4\%$

3 признака

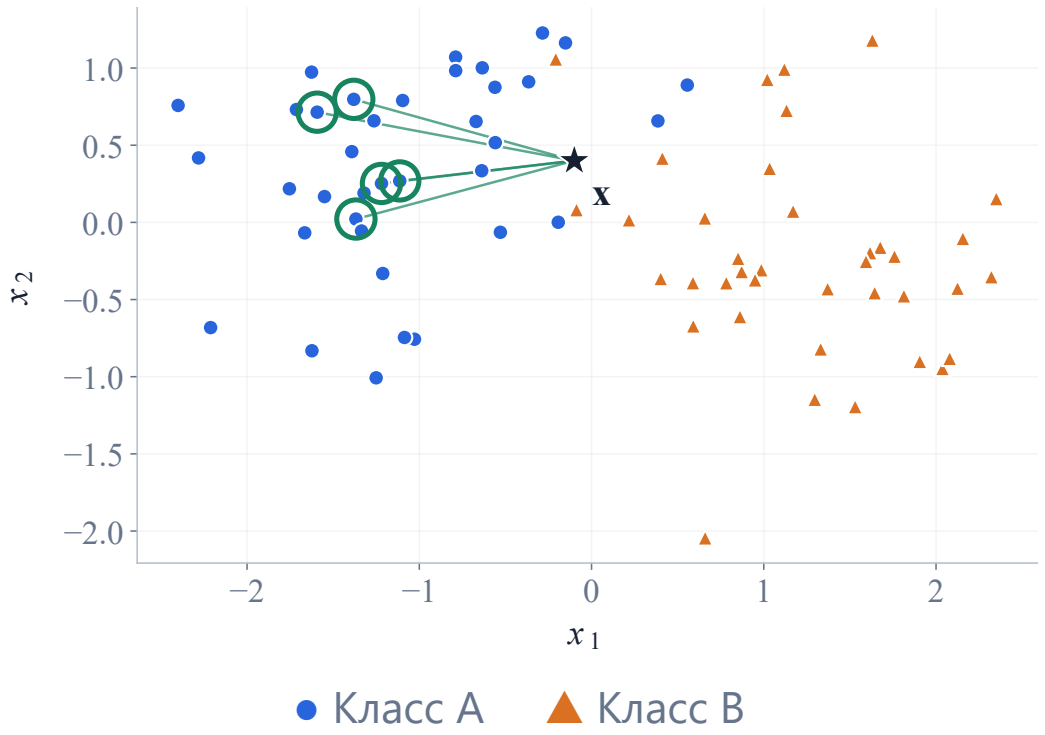


$0.2 \times 0.2 \times 0.2$

$r^3 = 0.008 \to 0.8\%$

Идеализируем: равномерное распределение в единичном пространстве; вся окрестность лежит внутри него.

Шумовые координаты меняют само соседство



2 полезные + 30 шумовых

Кольца и отрезки отмечают выбранных соседей.

На рисунке всегда показаны только первые две координаты.

Стандартизация меняет масштаб, но не устраняет нерелевантность признаков.

KD-tree и BallTree — структуры для ускорения поиска соседей. Их выигрыш зависит от геометрии и размерности данных.

Прогноз определяется соседством

- Класс выбираем по голосам ближайших обучающих объектов.
- Соседей определяют расстояние и масштаб признаков; число k выбираем по CV.
- Шумовые признаки и высокая размерность затрудняют поиск полезных соседей.

---

kNN уже работает с несколькими классами. Как расширить логистическую модель?

От kNN к многоклассовой постановке

kNN уже выбирает между A, B и C.
Как расширить бинарную логистическую регрессию?

$$Y \in \{1, \ldots, K\}$$

В многоклассовой классификации объекту соответствует **одна метка** из K взаимоисключающих вариантов.

Что должна выдавать многоклассовая модель

A · Оплата

Списание денег

B · Доставка

Движение заказа

C · Техподдержка

Ошибка сервиса

Для выбора класса достаточно оценок. В вероятностной модели хотим получить согласованный вектор классовых вероятностей:

$$x \longrightarrow (p_A(x), p_B(x), p_C(x)) \longrightarrow \hat{y}(x)$$

$$p_c(x) \geq 0, \qquad \sum_{c=1}^{K} p_c(x) = 1, \qquad \hat{y}(x) \in \arg\max_c p_c(x)$$

One-vs-Rest: A против остальных

Исходный класс	Смысл бинарного ответа	Новая метка
A · Оплата	A	1
B · Доставка	Не-A	0
C · Техническая проблема	Не-A	0

На тех же признаках обучаем знакомую бинарную модель: A против всех остальных.

Три самостоятельные бинарные модели

Истинный класс	A vs rest	B vs rest	C vs rest
A	1	0	0
B	0	1	0
C	0	0	1

Признаки объектов одни и те же; для каждой задачи меняем бинарные ответы.

Для K классов обучаем K отдельных моделей. Прогноз — класс с наибольшей бинарной оценкой:

$$\hat{y}(x) \in \arg\max_c \hat{p}_c^{bin}(x)$$

Три бинарные вероятности ещё не распределение

$$(\hat{p}_A^{bin}, \hat{p}_B^{bin}, \hat{p}_C^{bin}) = (0.8, 0.7, 0.3), \qquad \sum_c \hat{p}_c^{bin} = 1.8$$

У объекта один правильный класс. Можно ли считать эти числа его распределением?

$$\hat{p}_c^{OvR} = \frac{\hat{p}_c^{bin}}{\sum_j \hat{p}_j^{bin}} \quad \Rightarrow \quad (0.444, 0.389, 0.167)$$

Нормировка объединяет выходы. Параметры бинарных моделей всё ещё обучались независимо.

Что мы получили с помощью OvR

- Обучили K независимых бинарных классификаторов.
- По их оценкам выбираем один итоговый класс.
- Бинарные вероятности могут не давать сумму 1; нормировка не меняет способа обучения.

Можно ли обучить одну модель, в которой все классы учитываются совместно?

По одной линейной оценке для каждого класса

Один объект описан двумя признаками. Каждый класс получает свою строку коэффициентов.

$$s_A(x) = w_{A1}x_1 + w_{A2}x_2 + b_A,$$

$$s_B(x) = w_{B1}x_1 + w_{B2}x_2 + b_B,$$

$$s_C(x) = w_{C1}x_1 + w_{C2}x_2 + b_C.$$

Три оценки одного объекта — параметры одной многоклассовой модели. Найдём их по общей функции потерь.

Соберём коэффициенты в матрицу W

$$W = \begin{pmatrix} w_{A1} & w_{A2} \\ w_{B1} & w_{B2} \\ w_{C1} & w_{C2} \end{pmatrix}, \qquad b = \begin{pmatrix} b_A \\ b_B \\ b_C \end{pmatrix}$$

$$\underbrace{s(x)}_{K\times 1} = \underbrace{W}_{K\times p}\ \underbrace{x}_{p\times 1} + \underbrace{b}_{K\times 1}$$

Строка W — коэффициенты одного класса. Столбец W — влияние одного признака на все классовые оценки.

Посчитаем три оценки одного объекта

$$W = \begin{pmatrix} 1 & 0.5 \\ 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad b = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}, \quad x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

$$s_A = 1 \cdot 1 + 0.5 \cdot 2 = 2,$$
$$s_B = 1 \cdot 1 + 0 \cdot 2 = 1,$$
$$s_C = 0.$$

$$s(x) = Wx + b = \begin{pmatrix} 2 \\ 1 \\ 0 \end{pmatrix}$$

Один набор коэффициентов применяется ко всем объектам. Меняется x — меняются scores.

Экспоненты дают положительные веса классов

$$p_c = \frac{e^{s_c}}{\sum_j e^{s_j}} > 0$$

$$s_A > s_B \Rightarrow e^{s_A} > e^{s_B}$$

Нормируем положительные веса: сумма р равна 1.

Порядок оценок сохраняется.

$$\frac{p_A(x)}{p_B(x)} = \frac{e^{s_A(x)}}{e^{s_B(x)}} = e^{s_A(x) - s_B(x)}$$

$$\boxed{\log \frac{p_A(x)}{p_B(x)} = s_A(x) - s_B(x)}$$

Логарифм отношения вероятностей снова линейно связан с признаками.

Softmax: одна формула для всех классов

$$p_c(x) = \frac{e^{s_c(x)}}{\sum_{j=1}^{K} e^{s_j(x)}}$$

Свой числитель.
Общий знаменатель.

Класс	Score	Экспонента	Вероятность
A	2	7.389	0.665
B	1	2.718	0.245
C	0	1.000	0.090

Общий знаменатель: 11.107

Положительные вероятности суммируются в 1. Общий знаменатель связывает классы, но нормировка не гарантирует калибровку.

Прогноз — класс с наибольшей вероятностью: в этом примере A.

Cross-entropy одного многоклассового объекта

Модель выдала вероятности трёх классов. Истинный класс объекта — **B**.

A	**B · истинный класс**	C
≈ 0.665	**≈ 0.245**	≈ 0.090

$$L_i = -\log p_B \approx 1.408$$

$$\boxed{L_i = -\log p_{y_i}(x_i)}$$

Чем меньшую вероятность модель назначает правильному классу, тем больше значение функции потерь.

Одна целевая функция для всех коэффициентов

$$R_n(W, b) = -\frac{1}{n} \sum_{i=1}^{n} \log p_{y_i}(x_i; W, b)$$

Усредняем cross-entropy всех обучающих объектов.

$$(\hat{W}, \hat{b}) \in \arg\min_{W,b} R_n(W, b)$$

В одной минимизации участвуют все строки W и все свободные члены.

Что значит обучить многоклассовую модель

W, b → Линейные scores → Softmax → Средняя cross-entropy

Коэффициенты всех классов — параметры одной модели. Softmax связывает вероятности, поэтому общая функция потерь зависит от всего набора W,b.

Численный оптимизатор подбирает все параметры совместно. Можно добавить знакомый штраф на коэффициенты.

OvR: K отдельных бинарных задач.

Multinomial: одна совместная задача.

Далее: как оценить ошибки модели по отдельным классам?

Матрица ошибок в общей форме

Прогноз модели →

Истинный класс	A	B	C
A	C_{AA}	C_{AB}	C_{AC}
B	C_{BA}	C_{BB}	C_{BC}
C	C_{CA}	C_{CB}	C_{CC}

Строки — истинные классы.
Столбцы — прогноз модели.

$$TP_C = C_{CC}$$

$$FP_C = C_{AC} + C_{BC}$$

$$FN_C = C_{CA} + C_{CB}$$

Выбрали C положительным классом, остальные — отрицательными. Для любого другого класса делаем то же самое.

Одна матрица: 60 обращений в поддержку

Прогноз модели →

Истинный класс	A	B	C
A	38	2	0
B	4	10	1
C	2	2	1

Истинных объектов:
A — 40, B — 15, C — 5.

$$Accuracy = \frac{38 + 10 + 1}{60} = \frac{49}{60} \approx 0.817$$

Хорошо ли модель работает для каждого отдела?

Для класса C: кто TP, FP и FN?

Прогноз модели →

Истинный класс	A	B	C
A	38	2	0
B	4	10	1
C	2	2	1

Считаем C положительным классом.

$$TP_C = 1$$

$$FP_C = 0 + 1 = 1$$

$$FN_C = 2 + 2 = 4$$

Precision, recall и F1 для одного класса

Прогноз модели →

Истинный класс	A	B	C
A	38	2	0
B	4	10	1
C	2	2	1

$$Precision_C = \frac{1}{1+1} = 0.5$$

$$Recall_C = \frac{1}{1+4} = 0.2$$

$$F_{1,C} = \frac{2 \cdot 0.5 \cdot 0.2}{0.5 + 0.2} \approx 0.286$$

Precision: среди предсказанных C.

Recall: среди всех настоящих C.

Метрики по всем классам

Класс	Support	Precision	Recall	F1
A	40	0.864	0.950	0.905
B	15	0.714	0.667	0.690
C	5	0.500	0.200	0.286

Support — число объектов истинного класса: сумма его строки матрицы.

Общая accuracy ≈ 0.817 скрывает то, что найдена только пятая часть объектов C.

Macro: классы имеют одинаковый вес

$$F_1^{macro} = \frac{1}{K} \sum_{c=1}^{K} F_{1,c}$$

A · 1/3	B · 1/3	C · 1/3

$$F_1^{macro} \approx \frac{0.905 + 0.690 + 0.286}{3} \approx 0.627$$

C встречается в восемь раз реже A, но его вес в macro остаётся тем же.

Weighted: вес определяется долей класса

$$F_1^{weighted} = \sum_c \frac{n_c}{n} F_{1,c}$$

A · 40/60	B · 15/60	C

$$\frac{40 \cdot 0.905 + 15 \cdot 0.690 + 5 \cdot 0.286}{60} \approx 0.799$$

У крупных классов больше вес. Усредняем уже вычисленные классовые F1.

Одинаковые F1 — разные веса классов

Каждый класс весит 1/3

$$F_1^{macro} \approx 0.627$$

Вес пропорционален доле класса

$$F_1^{weighted} \approx 0.799$$

Значения F1 классов A, B, C остались прежними: 0.905, 0.690, 0.286. Изменились только веса при усреднении.

А если сначала объединить количества ошибок по всем классам и уже потом вычислить метрику?

Сначала объединяем TP, FP и FN

В macro и weighted сначала считали метрики классов. В micro сначала складываем количества по всем классам:

$$TP_{\text{all}} = \sum_{c=1}^{K} TP_c$$

$$FP_{\text{all}} = \sum_{c=1}^{K} FP_c$$

$$FN_{\text{all}} = \sum_{c=1}^{K} FN_c$$

Индекс all — итог по всем классам, а не имя отдельного класса.

Из этих общих количеств вычислим precision, recall и F1.

От одной ошибки — к общему подсчёту

$$n_{\text{correct}} \text{ — верных}, \qquad n_{\text{errors}} = n - n_{\text{correct}} \text{ — ошибок}$$

Правильный прогноз

Один TP своему классу.

$$TP_{\text{all}} = n_{\text{correct}}$$

Неправильный прогноз

Один FN истинному классу и один FP предсказанному.

$$FP_{\text{all}} = FN_{\text{all}} = n_{\text{errors}}$$

У каждого объекта одна истинная и одна предсказанная метка. В нашей матрице: 49 верных, 11 ошибок. Считаем по всем классам.

Почему micro совпадает с accuracy

Верный прогноз даёт один TP. Ошибка — один FN истинному и один FP предсказанному классу.

$$Precision^{micro} = \frac{TP_{\text{all}}}{TP_{\text{all}} + FP_{\text{all}}} = \frac{n_{\text{correct}}}{n}$$

$$Recall^{micro} = \frac{TP_{\text{all}}}{TP_{\text{all}} + FN_{\text{all}}} = \frac{n_{\text{correct}}}{n}$$

$$\boxed{Precision^{micro} = Recall^{micro} = F_1^{micro} = Accuracy}$$

В нашем примере: 49/60 ≈ 0.817. Micro отражает общий подсчёт верных и ошибочных ответов и не даёт отдельной информации о редких классах.

Для single-label multiclass с учётом всех классов. При равных precision и recall их F1 равно тому же числу.

Какую метрику выбрать для задачи?

Задача	Что важно узнать	Основной ориентир
Маршрутизация обращений	Какую долю направляем верно?	Accuracy при сопоставимой цене ошибок
NYC Taxi: четыре интервала длительности	Как различаем все четыре класса?	Macro-F1, метрики классов и матрица ошибок
Критичный редкий исход	Сколько находим? Скольким сигналам верим?	Recall и precision этого класса

Главные выводы

Регуляризация

Штраф меняет выбор коэффициентов. Силу ограничения проверяем на отложенных данных.

kNN и геометрия

Расстояния, масштаб признаков и число k определяют соседей и прогноз.

Кросс-валидация

Сравниваем всю процедуру на одинаковых фолдах. После выбора — refit на всех dev.

Многоклассовая задача

Различаем обучение и оценивание. Macro-F1 дополняем метриками классов и матрицей ошибок.

Фиксируем процедуру, проверяем на отдельном test и объясняем полученный результат.